

GPU Acceleration & ML/LLM Performance Improvement Plan

System: AI Governance Backend (`AI-Governance-version2` , branch `version-2` , HEAD `c02d40c`)
Prepared: 2 September 2026 | **Scope:** Analysis only — no code was modified
Basis: Source inspection, live introspection of the installed environment, and 1,394 recorded latency samples from `benchmark.Log`

Every number in this document was measured on this machine or read out of this repository. Where a statement is an inference rather than a direct measurement, it is labelled **INFERRED** and the supporting evidence is given. Where a number could not be measured, it is stated as unmeasured rather than estimated. Appendix A lists the exact commands and raw outputs behind every claim.

1. Executive Summary

The system is presently a **100 % CPU inference stack**. Nothing in it — not the LLMs, not the toxicity classifier, not the NER pipeline, not the embedding model — touches the GPU today. Moving to a GPU machine will help, but the seven findings below determine whether that move produces a 10× improvement or a 1.2× improvement.

#	Finding	Measured evidence	Consequence
1	The GPU path currently crashes. Ollama's CUDA backend cannot run on this box's driver/GPU combination.	<code>llama-server</code> process has terminated: <code>exit status 0xc0000409 ... CUDA error: the provided PTX was compiled with an unsupported toolchain</code> — reproduced 2026-09-02.	Blocks everything else. Must be resolved before any GPU work has meaning.
2	All LLMs are explicitly pinned to CPU by <code>Modelfile</code> , and the running models confirm it.	<code>Modelfile.cpu</code> / <code>Modelfile.tiny-cpu</code> : <code>PARAMETER num_gpu 0 . /api/ps</code> reports <code>size_vram: 0</code> and 100% CPU for both loaded models.	Decode is capped at 10.4 tok/s .
3	PyTorch is a CPU-only build , so <code>Detoxify</code> , <code>spaCy</code> and the embedding model cannot use a GPU even if one appears.	<code>torch 2.13.0+cpu</code> ; <code>torch.version.cuda = None</code> ; <code>cuda.is_available() = False</code> . <code>Dockerfile</code> installs from the CPU wheel index deliberately.	Encoder models stay on CPU regardless of hardware.
4	The LLM watchdog has no output cap. Its payload contains no <code>options</code> block at all.	<code>llm_watchdog.py:45-51</code> . Result: 26.2 % of all watchdog calls (96 of 367) hit the 120 s timeout ; <code>p50 = 57.7 s</code> .	Largest single source of wasted compute in the system.
5	Background watchdog calls queue in front of user-facing calls on the same single Ollama slot.	Foreground calls slower than 60 s overlap with 12.1 concurrent watchdog calls on average; calls under 30 s overlap with only 5.2 . Observed maximum foreground latency 285.6 s against a solo ceiling of 28.8 s.	Head-of-line blocking inflates user latency by up to ~10×.
6	The Presidio NLP engine silently discards <code>en_core_web_lg</code> . Both models are registered under language code <code>en</code> ; the second overwrites the first.	Engine reports one model under <code>en</code> , named <code>ner_bc5cdr_md</code> , whose only NER labels are <code>CHEMICAL</code> and <code>DISEASE</code> . Verified side-by-side: with the shipped config PERSON is never detected ; with <code>en_core_web_lg</code> alone it is detected at score 0.85.	Correctness defect plus a wasted model load. Fix before optimising.
7	LLM inference is ≥ 99 % of the wall clock . Everything else is noise by comparison.	Mean per call: LLM watchdog 64.9 s, initial LLM call 33.9 s, tool-feedback call 23.8 s — versus PII guardrail 61 ms and toxicity check 0.5–1.0 s.	Optimisation effort must be spent on the LLM tier first.

The single most important conclusion: the CPU-only Modelfiles are not a stylistic choice that can simply be reverted. They are a workaround for a GPU stack that *crashes on this hardware*. Section 3 covers this in full. Any plan that begins with “switch `config.json` to the GPU model tags” will fail with an HTTP 500 until the CUDA runtime problem is fixed.

2. Measured Baseline

2.1 Hardware and runtime as found

GPU	NVIDIA Quadro P1000 — 4,096 MiB total, compute capability 6.1 (Pascal), 1,369 MiB already consumed by desktop processes (VS Code, Edge WebView2)
Driver / CUDA	556.39 / CUDA 12.5
Host RAM available	~32.6 GB (from Ollama's own fit calculation)
Ollama	0.33.2
PyTorch	2.13.0+ cpu — CPU-only wheel, 6 threads
Other ML libs	transformers 5.14.1, spaCy 3.7.5, sciSpaCy 0.6.2, sentence-transformers 5.6.1
Accelerator libs	None installed: no onnxruntime, no optimum, no CTranslate2, no OpenVINO, no IPEX, no cupy, no accelerate
Server	uvicorn 0.51.0, single process, no <code>--workers</code> flag (<code>start_backend.bat</code>)

2.2 Model inventory (what is actually loaded)

Component	Model	Size / params	Precision	Device
Chat / watchdog / rule-suggestion LLM	phi4-mini-cpu (phi3 arch, 3.8 B, Q4_K_M)	3.1 GB resident	Q4_K_M	100 % CPU - size_vram 0
“Toxic mode” demo LLM	tinyllama-cpu (1.1 B, Q4_0)	735 MB resident	Q4_0	100 % CPU - size_vram 0
Toxicity classifier	Detoxify unbiased → RobertaForSequenceClassification, 12 layers, hidden 768	124.66 M params	fp32	cpu (library default, never overridden)
NER / PII	en_ner_bc5cdr_md (en_core_web_lg loaded then discarded)	—	fp32	CPU (spaCy has no GPU here — cupy absent)
RAG embeddings	all-MiniLM-L6-v2, max_seq 256	22.7 M params	fp32	cpu

2.3 Where the time goes — 1,394 samples from benchmark.log

All values in milliseconds. Percentiles computed over the full recorded history (2026-07-31 → 2026-08-29).

Component	n	min	p50	p90	p95	max	mean
LLM_WATCHDOG (background)	367	4,661	57,654	122,056	122,064	122,271	64,934
LLM_INITIAL_CALL	237	0	9,807	91,743	157,205	285,620	33,892
LLM_TOOL_FEEDBACK_CALL	35	4,654	5,001	70,487	102,219	123,259	23,774
EGRESS_TOXICITY_CHECK	237	0.3	186	2,452	4,435	15,862	966
INGRESS_TOXICITY_CHECK	281	0.4	62	950	2,168	13,627	545
EGRESS_PII_GUARDRAIL	237	0.3	29	138	201	943	61

Two things in that table need explaining before it can be read correctly:

- **The min = 0 entries are not real inferences.** 79 of 237 initial LLM calls (33.3 %) never reached Ollama at all — `demo_chat_agent` intercepts the strings `swiggy`, `all customer details`, `top 3 spenders` and `iban` in Python (`api.py:1147-1176`) and synthesises the answer. Likewise 40 of 518 toxicity samples (7.7 %) were recorded while the guard was disabled. **The real LLM p50 is worse than the table shows.**
- **The watchdog p95 of 122,064 ms is the client timeout, not a completion time.** `llm_watchdog.py:55` sets `timeout=120`. Those 96 calls burned up to two minutes of CPU each and returned nothing.

2.4 Raw generation throughput (measured today)

Measured against the live Ollama instance using the production watchdog system prompt (362 prompt tokens):

Configuration	Decode	Prefill (cold)	Prefill (warm)
phi4-mini-cpu, Q4_K_M, CPU	10.31 - 10.61 tok/s	201.7 tok/s	2,824 - 3,825 tok/s

Already correct — do not change: Ollama's KV prefix cache is working. The server log shows `cached n_tokens = 361` out of 362 prompt tokens on repeat calls, and prefill time collapsed from 1,795 ms to 94 ms between the first and second request. This works because the watchdog's system prompt is a byte-stable constant placed first. Section 6.1 explains where that property is being broken elsewhere.

3. Blocker: the GPU stack does not currently run

Before any of the recommendations below can be applied, this has to be resolved. It was reproduced directly on 2026-09-02 by unloading the CPU-pinned runners and requesting the GPU model tag `phi4-mini:3.8b`:

```
HTTP 500
{"error": "llama-server process has terminated: exit status 0xc0000409:
  The system detected an overrun of a stack-based buffer in this application.
  : CUDA error: the provided PTX was compiled with an unsupported toolchain."}
```

Ollama *did* detect the GPU and *did* plan an offload before crashing. From `server.log`:

```
inference compute id=0 library=CUDA compute=6.1 name=CUDA0 description="Quadro P1000"
  libdirs=ollama,cuda_v12 driver=12.5 total="4.0 GiB" available="3.3 GiB"

WARN llama-server discovery: could not determine compute capability for CUDA device -
  architecture filtering disabled for this device. If inference crashes, check that the
  CUDA backend supports this GPU.

common_params_fit_impl: - CUDA0 (Quadro P1000): 24 layers, 2293 MiB used, 1069 MiB free
load_tensors: offloaded 24/33 layers to GPU
load_tensors:   CUDA0 model buffer size = 1834.54 MiB
load_tensors:   CUDA_Host model buffer size = 1014.83 MiB
llama_kv_cache:   CUDA0 KV buffer size = 368.00 MiB
```

3.1 What this tells us

- 1. **The crash is a toolchain/architecture mismatch, not a memory problem.** “PTX compiled with an unsupported toolchain” means the shipped CUDA kernels were built for a newer driver than 556.39, and/or the CUDA 12 backend in Ollama 0.33.2 no longer emits SM 6.1 (Pascal) cubins — leaving only PTX that this driver cannot JIT. The explicit warning that Ollama could not even determine the compute capability of this device supports the second reading.
- 2. **Even if it ran, this GPU is too small for the current configuration.** Only 24 of 33 layers fit. 1,015 MiB of weights would stay in host memory and cross PCIe on every token. Partial offload of this ratio is frequently *slower* than pure CPU on a x8/x16 Pascal card. On this box the target model does not fit, period: 2.5 GB of Q4_K_M weights plus a 368 MiB KV cache against 3.3 GiB free — before the desktop takes its 1.37 GiB back.
- 3. **Two models are pinned resident simultaneously** (`keep_alive: -1`): 3.1 GB + 735 MB. On a 4 GB card they cannot co-reside. On the target GPU machine this must be sized deliberately.

3.2 Required actions, in order

#	Action	Why / acceptance test
3a	On the target GPU machine, verify compute capability ≥ 7.0 (Turing or newer) and a driver new enough for the CUDA runtime the chosen Ollama build ships. Do not assume the P1000 result generalises — but do not assume it doesn't either.	Acceptance: <code>/api/ps</code> shows <code>size_vram > 0</code> and 100% GPU for the chat model. This one check invalidates or validates the whole plan.
3b	Size VRAM against the <i>actual</i> footprint: model weights + KV cache + compute buffer, for <i>every</i> model you intend to keep resident, plus headroom for the desktop.	Measured here: weights 2,368 MiB, KV 512 MiB, compute 64 MiB for a single 4 K-context phi4-mini. Multiply the KV term by <code>OLLAMA_NUM_PARALLEL</code> .
3c	Only then switch <code>config.json</code> from the <code>*-cpu</code> tags to <code>phi4-mini:3.8b</code> / <code>tinylama:latest</code> . Both tags already exist locally.	<code>config.py</code> is already designed for this — file values and env vars both override, no rebuild needed. Nothing in the code needs to change.
3d	If the GPU cannot be made to work, the CPU optimisations in Sections 4–7 still apply and are individually worth more than the naive GPU move.	Items 4.1, 4.2, 5.1 and 6.2 are hardware-independent.

4. LLM Serving — highest impact first

4.1 **CRITICAL** Cap the watchdog's output length

`llm_watchdog.py` sends this payload:

```
payload = {
    "model": config.DEFAULT_LLM_MODEL,
    "messages": messages,
    "stream": False,
    "format": "json",
    "keep_alive": -1
} # <-- no "options" key at all
```

There is no `num_predict` and no `temperature`. By contrast every call inside `api.py` does set them (`num_predict 300`, `300` and `150` at lines 424, 573 and 1145; `temperature 0.0`). The watchdog therefore generates until the model decides to stop, at 10.4 tokens per second, under a 120-second ceiling.

Measured consequence: 96 of 367 calls (26.2 %) ran the full 120 s and were discarded. At 10.4 tok/s that is roughly 1,250 tokens of generation thrown away per timeout.

- Add `"options": {"num_predict": 256, "temperature": 0.0}`. The watchdog's output is a small JSON findings list; 256 tokens is generous.
- Setting `temperature 0.0` also makes the audit result reproducible — a governance requirement in its own right, independent of speed.
- Lower `timeout=120` to something consistent with the cap (a 256-token budget at 10.4 tok/s is ~25 s on CPU; far less on GPU). A timeout must be a safety net, not a routine outcome.

Expected effect (arithmetic, not measured): capping generation at 256 tokens bounds watchdog decode at ~25 s on the current CPU throughput, versus a measured mean of 64.9 s. This is the single cheapest change in this document.

4.2 **CRITICAL** Stop background watchdog calls from blocking user requests

Every governed endpoint schedules the watchdog as a FastAPI background task (`api.py:352, 371, 400, 1249`), which fires a second full LLM call *after* the response is returned. Ollama's default parallelism is one slot per model, and nothing in this repository sets `OLLAMA_NUM_PARALLEL` — a repository-wide grep for it returns no source matches.

Evidence for contention **INFERRED**, derived by reconstructing call intervals from `benchmark.log` (each row records an end timestamp and a duration, so the start is recoverable):

Foreground LLM call bucket	n	Mean overlapping watchdog calls
Slower than 60 s	52	12.08
Faster than 30 s	92	5.21

A solo 300-token generation at the measured 10.4 tok/s takes 28.8 s. The observed maximum is 285.6 s — very close to ten times the solo figure, consistent with queueing behind roughly ten pending jobs. The correlation above and that ratio together make queueing the most plausible explanation for the p95/p99 tail; it is not directly instrumented, so it is labelled inferred.

Options, roughly in order of preference:

- **Give the watchdog its own capacity.** Set `OLLAMA_NUM_PARALLEL` (2–4 on a GPU with the VRAM for the extra KV caches — budget the measured 512 MiB KV per 4 K slot). This is a server-side environment variable, no code change.
- **Bound the queue.** Today an unbounded number of background tasks can pile up. A semaphore or a small worker queue in front of `run_watchdog_task` would cap it. Dropping a watchdog check under load is preferable to a 285-second user response — but that is a governance policy call, not purely an engineering one, so it needs a decision from whoever owns the compliance requirement.

- **Skip work that cannot find anything.** Cache watchdog verdicts keyed on `sha256(raw_text)`. The load-test client draws messages at random with replacement from a fixed bank, so repeated identical texts are common in benchmarking, and the demo flows re-emit identical DB payloads. A hash cache converts those to zero-cost.
- **Separate the models.** If `OLLAMA_MAX_LOADED_MODELS` allows and VRAM permits, running the watchdog on a smaller model than the chat path removes the contention entirely. Neither `OLLAMA_MAX_LOADED_MODELS` nor `OLLAMA_KEEP_ALIVE` is set anywhere in the repository today.

4.3 **HIGH** Enable streaming for user-facing calls

All three call sites use `"stream": False` (`api.py:423`, `api.py:571`, `api.py:1144`, `llm_watchdog.py:49`). The user therefore waits for the entire generation before seeing anything.

This does not reduce total compute, but it changes perceived latency from “full decode” to “time to first token” — measured warm prefill here is 94–128 ms against a 12–16 s full response.

Important caveat, and it is a real one: the egress PII guardrail and the egress toxicity check both operate on the *complete* model output. You cannot stream raw tokens to a user and still guarantee masking. Streaming is only safe if you either (a) stream into a buffer and mask on sentence/chunk boundaries with a lookback window large enough to cover the longest maskable entity, or (b) stream only a “thinking” indicator and release the masked text at the end. Option (a) is the real win and is genuine engineering work; option (b) is cosmetic. Do not ship (a) without a test proving no entity can straddle a chunk boundary unmasked.

4.4 **HIGH** Eliminate the second LLM round trip where it does nothing

The tool-call flow costs a full second inference (`LLM_TOOL_FEEDBACK_CALL`: p50 5.0 s, p95 102.2 s, mean 23.8 s). In two of the three branches, that second call asks the model to *echo a string the backend already has*:

```
# api.py:1200 - swiggy branch
"content": f"Here is the database result: {raw_data}. Output EXACTLY this sentence and
           nothing else: 'The GPS coordinates for the delivery driver are 48.8584 N, 2.2945 E.'"
```

```
# api.py:1206 - iban branch
"content": f"Here is the database result: {raw_data}. Output EXACTLY this sentence and
           nothing else: '{raw_data}'"
```

These two branches spend a complete LLM inference producing text that is already a Python literal in the request. For the demo they can return the string directly and still exercise the full guardrail path — the governance demonstration is unaffected, because the guardrail runs on the string either way.

For the third branch (the JSON customer schema), the second call is doing real work, but note that `phi4-mini:3.8b` advertises a `tools` capability (`ollama show`). The application instead hand-rolls tool calling with a `<FETCH_DB:ID>` sentinel and a regex (`api.py:1181`). Native tool calling would not remove the second round trip — that is inherent to tool use — but it would remove the routing failures that currently forced the hardcoded keyword interception at `api.py:1147–1176`, which the source itself explains as a workaround: “*Small models (like phi4-mini) struggle with complex routing instructions.*”

4.5 **MEDIUM** Set `num_ctx` explicitly

A repository-wide search for `num_ctx` returns zero matches. The running defaults, from `/api/ps`, are 4,096 for `phi4-mini-cpu` and 2,048 for `tinyl llama-cpu`. The model's native context is 131,072.

- **On GPU this becomes a VRAM decision, not a capability decision.** KV cache scales linearly with `num_ctx` × `num_parallel`. Measured here: 368–512 MiB of KV for a single 4 K slot. Leaving it at the default while raising parallelism will silently push layers back to the host and destroy the speedup you are trying to buy.
- Set it to the smallest value that fits the real traffic. The largest observed prompt in the server log is 515 tokens; the watchdog's is 362. 2,048 would cover current usage with room to spare, and would roughly halve the KV budget versus 4,096.

- The server log shows `truncated = 0` on recent requests, so nothing is currently being cut — but there is no guard in the code against a long DB payload silently overflowing the window.

4.6 MEDIUM Ollama server-level tuning

None of the following are set anywhere in this repository. All are environment variables on the Ollama server process, requiring no application change:

Variable	Relevance here
<code>OLLAMA_NUM_PARALLEL</code>	Directly addresses the head-of-line blocking in 4.2. Costs one KV cache per slot — budget from the measured 368–512 MiB figure.
<code>OLLAMA_MAX_LOADED_MODELS</code>	The app uses two distinct models (<code>DEFAULT_LLM_MODEL</code> and <code>TOXIC_LLM_MODEL</code>). If this is below 2 on the GPU box, every mode switch in <code>/demo_chat</code> triggers a model swap. Currently both are pinned resident via <code>keep_alive: -1</code> , so on CPU this is a non-issue; on a VRAM-constrained GPU it becomes one.
<code>OLLAMA_FLASH_ATTENTION</code>	Reduces attention memory and improves throughput on supported architectures. Not applicable to the Pascal card here; likely applicable on the target machine. Unmeasured — benchmark it, do not assume.
<code>OLLAMA_KV_CACHE_TYPE</code>	Quantised KV (q8_0/q4_0) trades a small quality cost for a large KV memory reduction, which buys back context length or parallel slots. Relevant precisely because VRAM is the binding constraint. Unmeasured.

4.7 LOW / CONDITIONAL Consider a different serving runtime

Ollama is a good fit for a single-user desktop demo. It is not a throughput-oriented server. If the target deployment must handle genuine concurrency, runtimes with continuous batching (vLLM, TGI, SGLang, or `llama.cpp`'s server with a larger batch) amortise weights across concurrent requests instead of serialising them. That is a re-platforming decision with real cost, and the measured evidence here does not yet justify it — the current bottleneck is that the GPU is not being used at all, not that Ollama is scheduling badly. **Revisit only after Sections 3 and 4.1–4.2 are done and re-measured.**

5. Encoder Models — Detoxify, spaCy, Embeddings

These are collectively 1–3 % of current wall-clock time. They matter for two reasons: they will become a visible fraction once the LLM tier is fixed, and one of them contains a correctness defect.

5.1 **CRITICAL** The Presidio NLP engine discards `en_core_web_lg`

`api.py:49–57` registers two models under the same language code:

```
"models": [  
    {"lang_code": "en", "model_name": "en_core_web_lg"},      # General purpose  
    {"lang_code": "en", "model_name": "en_ner_bc5cdr_md"}    # Medical entities  
]
```

Presidio keys its model map by language code. The second entry overwrites the first. Verified directly:

```
loaded models dict keys: ['en']  
lang en -> meta name: ner_bc5cdr_md  
          pipes: ['tok2vec', 'tagger', 'attribute_ruler', 'lemmatizer', 'parser', 'ner']  
  
en_ner_bc5cdr_md NER labels: ('CHEMICAL', 'DISEASE')  
en_core_web_lg  NER labels: ('CARDINAL', 'DATE', ..., 'ORG', 'PERSON', ...)
```

Then, through the analyzer exactly as `api.py` builds it:

```
text: "John Smith was diagnosed with diabetes. Card 4111 1111 1111 1111. Email john.smith@acme.com"  
  
AS-CONFIGURED (both models): [CREDIT_CARD '4111 1111 1111 1111', EMAIL_ADDRESS 'john.smith@acme.com']  
en_core_web_lg ONLY:        [CREDIT_CARD ..., EMAIL_ADDRESS ..., PERSON 'John Smith' @0.85]
```

PERSON is in `ACTIVE_ENTITIES` and has a custom masking operator (`mask_person_name`, `api.py:252`), but the configured pipeline can never produce it. Names are not being masked by the NER path. This is a governance correctness defect first and a performance issue second — `en_core_web_lg` is loaded (measured 1.43 s and its full weight in RAM) and then thrown away.

The fix is a design decision, not a one-liner: Presidio's one-model-per-language structure means running two NER models requires either a merged spaCy pipeline (adding the bc5cdr NER component as a second, renamed `ner` pipe on the `en_core_web_lg` pipeline) or two engines and a merge of their results. **Whichever is chosen, re-measure — this will make the guardrail slower, because it will finally be doing the work it was supposed to be doing.**

5.2 **HIGH** Detoxify is pinned to fp32 CPU

`toxicity_guard.py:16` is `_model = Detoxify('unbiased')`. The library signature is `Detoxify(model_type, checkpoint, device="cpu", ...)` — the `device` argument is never supplied, so on a GPU machine this 124.66 M-parameter RoBERTa will still run on CPU.

Measured on CPU: 32.2 ms for a short input, 292.4 ms for a long one. Recorded production means are much higher (ingress 545 ms, egress 966 ms, egress max 15.9 s) — the gap is CPU contention with the concurrently running Ollama process, which is a further argument for getting the LLM off the CPU.

Improvements, in order of value:

1. **Pass `device`**. Wire it to config the same way model names already are (`config.py` is built for exactly this pattern), defaulting to CPU so nothing regresses on the current box.
2. **Use fp16 on GPU**. The model is fp32 today. Half precision roughly halves memory traffic on tensor-core hardware. **Not applicable to the Pascal card here** (compute 6.1 has no tensor cores); applicable to the target machine. Validate the scores against the current thresholds afterwards — the thresholds in `pii_rules.json` are tuned at 0.4/0.5 and a precision change shifts borderline scores.

3. **Batch the two calls that already exist.** `/demo_chat` and `/chat` each run an ingress check and an egress check as two separate single-sample forward passes. Detoxify's `predict` already accepts a list and handles batched output. They cannot be merged in the current control flow (ingress must gate before the LLM runs), but micro-batching *across concurrent requests* is the real GPU-era win: a batch of 8 costs barely more than a batch of 1 on a GPU, and the endpoints are synchronous `def` functions running in uvicorn's thread pool, so several are genuinely in flight at once.
4. **Consider `unbiased-small`.** The library ships an ALBERT-based variant (`unbiased-albert`) under the same API. Substantially cheaper. This trades accuracy for speed on a content-safety control, so it needs a validation run against your test cases before adoption — the scores drive alarm generation.
5. **Export to ONNX Runtime / TensorRT.** Nothing of the sort is installed today (no onnxruntime, no optimum). For a fixed-shape 12-layer encoder this is a well-trodden path and typically the largest single-model speedup available, but it is real work and should follow items 1–3.

Correctness note found while measuring: the tokenizer's `model_max_length` is 512 and `predict` calls it with `truncation=True`. Any text beyond ~512 tokens is **silently unscanned for toxicity**. This is why the long-input measurement was only 292 ms — it stops at the cap. If long AI outputs must be fully covered, the text needs chunking with a sliding window and score aggregation, which *adds* cost. Flagging it because a governance control that silently stops at 512 tokens is a finding in its own right.

5.3 MEDIUM spaCy pipeline is running components Presidio does not use

The loaded pipeline is `['tok2vec', 'tagger', 'attribute_ruler', 'lemmatizer', 'parser', 'ner']`. Presidio's `SpacyRecognizer` consumes entities; the tagger, attribute ruler, lemmatizer and dependency parser are not needed for that path. Measured:

Pipeline	ms / document
Full pipeline as loaded	5.04
<code>exclude=['tagger', 'attribute_ruler', 'lemmatizer', 'parser']</code>	3.72

A 26 % reduction on the NER stage, at zero quality cost *for the NER path*. Verify first that no Presidio recognizer in your registry relies on lemmas or POS tags for context enhancement — Presidio's context-word mechanism, which the `AadhaarRecognizer` depends on for its score bump, uses the surrounding tokens. Test before adopting.

Two related notes:

- **spaCy on GPU is not automatic and probably not worth it here.** It requires `cupy` (not installed) and `spacy.require_gpu()`. These are CNN/tok2vec models operating on short single documents; at 3.7–5.0 ms per document, per-call GPU dispatch overhead would likely eat the gain. It only pays if you batch through `nlp.pipe()`, which the single-document request path does not do. **Low priority.**
- **Entity count does not affect analyzer cost.** Measured 6.40 ms for 3 entities and 6.49 ms for 7 — the NLP pass dominates, and narrowing `ACTIVE_ENTITIES` will not speed anything up.

5.4 LOW Embedding model / RAG

Measured: `all-MiniLM-L6-v2`, 22.7 M params, CPU, 11.93 ms per query encode, 12.78 s to load at startup. The corpus is a single file, `governance_policies/data_standards.md`, **1,003 bytes**.

- At this corpus size the retrieval is effectively free and a GPU would be pure overhead. Pass a `device` parameter for consistency, but expect nothing from it.
- `retrieve_relevant_policy` is called once per `/sandbox_suggest_rule` request — not in the chat hot path — so its 12 ms never appears in the latency table.
- The one real inefficiency: `rag_engine.py:44-45` re-runs `load_knowledge_base()` whenever `knowledge_texts` is empty. If the policies directory is ever missing, this re-attempts the whole load on every single call. Guard it with a "load attempted" flag rather than an emptiness check.
- Corpus embeddings are computed at startup and cached correctly — that part is already right.

6. Prompt-Level Improvements

Prompt engineering is a latency lever here because decode is the bottleneck: at 10.4 tok/s on CPU, every output token costs ~96 ms. Input tokens are much cheaper (2,800–3,800 tok/s warm prefill), so the priority order is *output length first, input structure second*.

6.1 Protect the KV prefix cache — one place is already breaking it

The measured evidence that prefix caching works is in Section 2.4: `cached_n_tokens = 361` of 362. That happens because the watchdog's system prompt is a module-level constant, identical byte-for-byte on every call, and placed first. `sandbox_suggest_rule` breaks this. At `api.py:537`, the system prompt is an f-string that interpolates the live entity list:

```
CRITICAL: The existing entities in our rule engine are: {existing_entities}.
...
{rag_instruction}      # RAG-retrieved policy text, also interpolated into the SYSTEM prompt
```

Both `existing_entities` and `rag_instruction` change between calls, and both sit inside the *system* message — the prefix. Any change invalidates the cache from that point onward. The fix is ordering, not content: keep the invariant instruction block first and byte-stable, and move all volatile content (entity list, retrieved policy, the snippet under analysis) into the user message that follows. Content is unchanged; only its position moves.

6.2 Shrink what the models are asked to write

The watchdog's output schema (`llm_watchdog.py:20–30`) requires, per finding:

```
{"type": "CATEGORY", "value": "...", "reason": "why it's sensitive"}
```

Consider what happens to each field downstream:

Field	Downstream use	Assessment
value	<code>diff_engine.run_diff</code> — the substring comparison against Presidio's masked spans. This is the mechanism.	Essential.
type	<code>generate_alarm</code> — mapped to a notification category via <code>pii_rules.json</code> keyword matching.	Essential, but could be constrained to a fixed enum instead of free text, which shortens it and makes category mapping deterministic.
reason	Stored in the alarm and shown to a reviewer. Purely explanatory.	Free-form prose, generated at 96 ms/token, on the critical path of a background audit. Making it optional — or generated only for alarms a human actually opens — removes real decode time.

The system prompt itself is 1,307 characters. The full request measured at 362 prompt tokens. That is cheap (~95 ms warm) and **not** where the time goes — resist the temptation to trim the instructions, which is where the accuracy lives. Trim the *output*.

6.3 Make the watchdog's instructions carry their own budget

The prompt currently says nothing about how much to produce. Combined with the missing `num_predict` (Section 4.1), that is how a 120-second timeout happens. Two complementary changes:

- State a hard limit in the prompt (“report at most N findings; if more exist, report the N most severe”). A prompt-level cap and a `num_predict` cap fail differently — the first yields valid truncated JSON, the second yields *invalid* JSON that `llm_watchdog.py:62` then logs as a decode failure. You want both, with the prompt limit binding first.

- `format: "json"` is already set, which constrains the grammar. Newer Ollama versions accept a full JSON *schema* rather than the string `"json"`. A schema with `maxItems` on the findings array enforces the budget at the sampler, which is stronger than any instruction. Worth testing against Ollama 0.33.2's exact support.

6.4 Do not send the model work it will fail at

`api.py:1145–1146` documents the current state of affairs candidly:

```
# Small models (like phi4-mini) struggle with complex routing instructions.
# For the sake of the demo, we explicitly intercept the known demo keywords.
```

The measured result is that 33.3 % of “LLM calls” are not LLM calls. That interception is fast, but it means the routing prompt at `api.py:1119–1130` — with its three worked examples — is only exercised on the remaining 66.7 %, unvalidated for anything outside the demo script. Two honest options:

- Use the model's native `tools` capability for routing (it is advertised by `phi4-mini:3.8b`). Constrained tool-call decoding is far more reliable than asking a 3.8 B model to emit a bare sentinel token, and it is *fewer* output tokens.
- Or keep deterministic routing and be explicit that it is deterministic — a keyword router in front of the LLM is a legitimate and much faster design. What should not persist is a routing prompt that is believed to work but is bypassed for the cases anyone actually tests.

6.5 Smaller prompt-level items

- **Temperature.** Set to 0.0 in all three `api.py` call sites; **absent in the watchdog**, which therefore samples at the model default. For a compliance auditor this is both a speed issue (sampling produces longer, more variable output) and a reproducibility issue.
- **The second-hop system-prompt rewrite** (`api.py:1194`) mutates `messages[0]["content"]` in place before the follow-up call. This is deliberate and the comment explains why, but it also guarantees a prefix-cache miss on the second hop — the system message changed. Appending a new instruction instead of rewriting the first message would preserve the cached prefix.
- **num_predict: 300 for chat** (`api.py:424, 1145`) is 28.8 s of decode at current CPU throughput. On GPU this ceases to matter; on CPU it is the difference between a usable and an unusable demo. Size it to the shortest useful answer.
- **The JSON schema is embedded as a formatted literal** in the tool-feedback message (`api.py:1213–1222`) with indentation and newlines. Minifying it saves a handful of prompt tokens — negligible, listed only for completeness.

7. Application-Level Issues That Will Surface Once the LLM Is Fast

These are currently invisible — they are milliseconds hiding behind 60-second LLM calls. When the LLM tier is fixed, they become the profile.

7.1 `pii_rules.json` is re-read from disk repeatedly per request

Distinct synchronous file reads on a single `/demo_chat` request:

Call site	Purpose
<code>load_toxicity_settings()</code> — <code>api.py:216</code>	Once per ingress toxicity check
<code>load_toxicity_settings()</code> — <code>api.py:216</code>	Again per egress toxicity check
<code>demo_chat_agent</code> — <code>api.py:1258</code>	Guardrail enable/disable check
<code>run_watchdog_task</code> — <code>api.py:325</code>	Watchdog enable check (background)
<code>diff_engine.generate_toxicity_alarm</code> — twice, lines 161 and 192	Watchdog-enabled check, then subscriber routing
<code>diff_engine.generate_alarm</code> — twice, lines 50 and 82	Category mappings, then subscriber routing

Four to eight reads and JSON parses of the same small file per request, all inside a single-process server. Cache it with an mtime check; the file already has an explicit reload path (`reload_presidio_engine`) for when rules change.

7.2 Synchronous file writes on the request path

- `toxicity_monitor.log` — one open/append/close per toxicity check (`api.py:245–249`). Currently 196 KB.
- `benchmark.log` — one open/append/close per metric, four to six metrics per request.
- `governance_audit.json` — append per transaction. **This one is already correct** and the code comment explains exactly why JSON Lines was chosen over an array rewrite. Good.
- `alarms.json` + `alarms_archive.json` — `save_alarm` (`diff_engine.py:29–38`) does `load` → `insert(0)` → `dump` on **both** files for every alarm. That is O(n) per alarm on a file that only grows; the archive is at 23 entries / 19 KB today and never shrinks. Under load-test volumes this becomes a measurable serialisation point.

7.3 `/sandbox_test_rule` constructs a whole new `AnalyzerEngine` per request

`api.py:595` is `sandbox_analyzer = AnalyzerEngine()` inside the request handler. Measured cost: **1.24–1.34 seconds** per call, loading 17 recognizers and a default spaCy pipeline each time, purely to test one regex. A module-level singleton whose registry is swapped per request, or simply testing the regex with Python's `re` module for the pattern-only case, removes essentially all of it.

7.4 Single `uvicorn` worker

`start_backend.bat` launches `uvicorn` with no `--workers` flag. All endpoints are synchronous `def` functions, so FastAPI runs them in the `anyio` thread pool — which does give real concurrency for the torch and spaCy calls, since those release the GIL. But there is exactly one process holding one copy of every model.

Do not reflexively add workers. Each worker is a full independent copy of Detoxify (124.66 M params), `en_ner_bc5cdr_md`, and MiniLM. On a GPU box that means N copies of the weights in VRAM, competing with the LLM for the same memory that Section 3.1 already shows is the binding constraint. The correct pattern for a GPU deployment is one process with batched inference (Section 5.2 item 3), not N processes each doing single-sample inference. Measure before scaling out.

7.5 PyTorch thread configuration

`torch.get_num_threads()` reports 6, and nothing in the repository sets it. While the LLM runs on CPU, PyTorch and Ollama are competing for the same cores — the most likely explanation for why a toxicity check measured at 32 ms standalone shows a 15.9 s maximum in production. Moving the LLM to GPU largely resolves this by itself; if the LLM stays on CPU, explicitly partitioning threads between the two is worth testing.

8. Prioritised Roadmap

Ordered by measured impact per unit of effort. Items in Phase 0 are prerequisites — the later phases are not meaningfully verifiable until they are done.

#	Action	Effort	Risk	Expected effect and how to confirm it
PHASE 0 — Unblock (nothing below is verifiable until these are done)				
0.1	Confirm the target GPU machine's compute capability and driver actually run Ollama's CUDA backend	Low	—	Gate for everything. Confirm: <code>/api/ps</code> shows <code>size_vram > 0</code> .
0.2	Size VRAM: weights + KV × parallel slots + compute buffer + desktop headroom	Low	Low	Prevents silent partial offload, which measured <i>slower</i> than CPU here.
0.3	Switch <code>config.json</code> to the GPU model tags	Trivial	Low	No code change — <code>config.py</code> already supports it. Both tags exist locally.
0.4	Install a CUDA build of PyTorch on the GPU machine	Low	Med	Without it Detoxify/spaCy/MiniLM stay on CPU forever. Note the <code>Dockerfile</code> deliberately installs the CPU wheel — that decision must be revisited for a GPU image.
PHASE 1 — Highest return, hardware-independent				
1.1	Add <code>num_predict + temperature: 0</code> to the watchdog payload	Trivial	Low	Targets the 26.2 % timeout rate directly. Confirm: timeout count in <code>benchmark.log</code> falls to ~0.
1.2	Bound / decouple background watchdog concurrency	Med	Med	Targets the p95 = 157 s and max = 285.6 s tail. Confirm: re-run the overlap analysis in §4.2 and check the correlation flattens.
1.3	Fix the Presidio dual-model overwrite (§5.1)	Med	High	Correctness, not speed — expect the guardrail to get slower because it will start doing PERSON detection. Confirm with a PERSON test case.
1.4	Remove the two echo-only second LLM hops (§4.4)	Low	Low	Eliminates a full inference (p50 5.0 s, p95 102 s) from two demo paths.
1.5	Cache <code>pii_rules.json</code> with an mtime check	Low	Low	Removes 4–8 disk reads per request.
PHASE 2 — GPU exploitation				
2.1	Pass <code>device</code> to Detoxify and SentenceTransformer, config-driven	Low	Low	The only way these models ever see the GPU.
2.2	Set <code>num_ctx</code> explicitly; tune <code>OLLAMA_NUM_PARALLEL</code> / <code>MAX_LOADED_MODELS</code>	Low	Med	Buys parallel slots within the VRAM budget. Watch <code>/api/ps</code> for layers falling back to host.
2.3	fp16 for Detoxify on GPU	Low	Med	Re-validate against the 0.4/0.5 thresholds in <code>pii_rules.json</code> — a precision change moves borderline scores.
2.4	Micro-batch Detoxify across concurrent requests	Med	Med	Where the GPU actually pays for an encoder. Batch of 8 ≈ cost of batch of 1.
2.5	Trim unused spaCy pipes (§5.3)	Low	Med	Measured 5.04 → 3.72 ms/doc. Verify context-word scoring (Aadhaar) still works.
2.6	Test <code>OLLAMA_FLASH_ATTENTION</code> / <code>KV_CACHE_TYPE</code>	Low	Low	Unmeasured here. Benchmark on the target hardware; do not assume.
PHASE 3 — Structural, only if measurement justifies it				
3.1	Prompt restructuring for prefix-cache stability (§6.1) and output-size reduction (§6.2)	Med	Med	Requires re-validating watchdog detection quality against <code>test_cases.json</code> .

3.2	Streaming with chunk-boundary-safe masking (§4.3)	High	High	Do not ship without a test proving no entity can straddle a chunk boundary unmasked.
3.3	ONNX Runtime / TensorRT export for Detoxify	High	Med	Typically the largest single-model win; nothing is installed for it today.
3.4	Watchdog result caching by input hash	Low	Med	Needs a policy decision on whether a cached compliance verdict is acceptable.
3.5	Alternative serving runtime with continuous batching (§4.7)	High	High	Only revisit after Phases 0–2 are measured. Current evidence does not justify it.

9. What This Analysis Does Not Establish

Stated explicitly so the plan is not read as promising more than it measured:

- **No GPU speedup figure is quoted anywhere in this document.** The GPU path crashed on this hardware, so no valid GPU-vs-CPU comparison could be produced. An initial A/B run appeared to show identical throughput — that result was *discarded*, because the server log showed Ollama had routed the request to the already-running CPU runner rather than starting a GPU one. Any speedup number must be measured on the target machine.
- **The queueing explanation in §4.2 is inference, not instrumentation.** It rests on a strong correlation (12.08 vs 5.21 overlapping calls) and a consistent ratio (285.6 s observed against a 28.8 s solo ceiling). It is well-supported, but confirming it requires per-request queue-depth instrumentation that does not exist today.
- **Flash attention and quantised KV cache were not benchmarked.** Neither is applicable to the Pascal card present here.
- **Detoxify's production means (545 ms / 966 ms) exceed its standalone measurement (32 ms / 292 ms) by a wide margin.** CPU contention with Ollama is the most likely cause and is consistent with §7.5, but this was not isolated experimentally.
- **The `frontend/` directory was not examined.** The question was scoped to the ML and LLM tiers.

Appendix A — Evidence Log

Every measurement was taken on 2026-09-02 against the working tree at commit `c02d40c`. Runtime state was restored afterwards: both CPU models were re-pinned with `keep_alive: -1` and `ollama ps` was confirmed to match its original output. No repository file was modified.

A.1 Environment

```
$ .venv/Scripts/python.exe -c "import torch; print(torch.__version__, torch.version.cuda,
torch.cuda.is_available())"
2.13.0+cpu    None    False

$ nvidia-smi --query-gpu=name,compute_cap,driver_version,memory.total,memory.used --format=csv
Quadro P1000, 6.1, 556.39, 4096 MiB, 1369 MiB

$ ollama --version
ollama version is 0.33.2
```

A.2 Loaded model state (unmodified)

```
$ curl -s http://localhost:11434/api/ps
phi4-mini-cpu:latest size 3,094,704,290 size_vram 0 context_length 4096
tinyllama-cpu:latest size 735,743,835 size_vram 0 context_length 2048

$ ollama show phi4-mini-cpu --parameters
num_gpu 0
```

A.3 Throughput measurement (production watchdog prompt, 362 tokens)

CPU warmup	wall 16,203 ms	eval 127 tok	10.31 tok/s	prefill 201.7 tok/s
CPU as-shipped	wall 16,323 ms	eval 150 tok	10.59 tok/s	prefill 3825.1 tok/s
CPU + num_predict=150	wall 14,077 ms	eval 126 tok	10.59 tok/s	prefill 2824.8 tok/s

server.log: slot operator(): cached n_tokens = 361, memory_seq_rm [361, end)
(prefill 1794.8 ms cold -> 94.6 ms warm)

A.4 Detoxify

```
arch: RobertaForSequenceClassification | model_type roberta | hidden 768 | layers 12
params: 124.657936 M | dtype torch.float32 | device cpu | tokenizer model_max_length 512
classes: toxicity, severe_toxicity, obscene, identity_attack, insult, threat, sexual_explicit
short input predict: 32.18 ms
long input predict: 292.41 ms (truncated at 512 tokens)
```

A.5 Presidio / spaCy

```
NlpEngineProvider(models=[en_core_web_lg, en_ner_bc5cdr_md])
-> nlp.keys() = ['en']; nlp['en'].meta['name'] = 'ner_bc5cdr_md'
-> pipes: tok2vec, tagger, attribute_ruler, lemmatizer, parser, ner

en_ner_bc5cdr_md labels: ('CHEMICAL', 'DISEASE')
en_core_web_lg labels: ('CARDINAL', 'DATE', 'EVENT', 'FAC', 'GPE', ..., 'PERSON', ...)

analyze("John Smith was diagnosed with diabetes. Card 4111 1111 1111 1111. Email john.smith@acme.com")
as-configured : CREDIT_CARD, EMAIL_ADDRESS
lg-only       : CREDIT_CARD, EMAIL_ADDRESS, PERSON 'John Smith' @0.85

pipeline cost: full 5.04 ms/doc ; tok2vec+ner only 3.72 ms/doc
analyzer cost: 6.40 ms (3 entities) ; 6.49 ms (7 entities)
model load : en_core_web_lg 1.43 s ; en_ner_bc5cdr_md 7.59 s
fresh AnalyzerEngine(): 1.34 / 1.24 / 1.28 s, 17 recognizers
```

A.6 Embeddings

```
all-MiniLM-L6-v2: load 12.78 s | device cpu | max_seq 256 | 22.7 M params
encode: 11.93 ms per query
corpus: governance_policies/data_standards.md - 1,003 bytes (single file)
```

A.7 Log-derived statistics (benchmark.log, 1,394 rows)

```
LLM_WATCHDOG      n=367  timeouts (>119 s): 96 = 26.2 %
LLM_INITIAL_CALL  n=237  bypassed LLM (<50 ms): 79 = 33.3 %
TOXICITY (both)   n=518  guard disabled (<5 ms): 40 = 7.7 %
```

Overlap analysis (start times reconstructed from end timestamp minus duration):

```
LLM_INITIAL_CALL > 60 s : n=52, mean concurrent LLM_WATCHDOG = 12.08
LLM_INITIAL_CALL <= 30 s: n=92, mean concurrent LLM_WATCHDOG = 5.21
solo ceiling: 300 tok / 10.4 tok/s = 28.8 s ; observed max = 285.6 s
```

A.8 GPU crash reproduction

```
$ POST /api/chat {"model":"phi4-mini:3.8b", ...} # after unloading the CPU-pinned runners
HTTP/500
{"error": "llama-server process has terminated: exit status 0xc0000409: The system detected an overrun of a stack-based buffer in this application. ... : CUDA error: the provided PTX was compiled with an unsupported toolchain."}
```

server.log (planning stage, before the crash):

```
common_params_fit_impl: - CUDA0 (Quadro P1000): 24 layers, 2293 MiB used, 1069 MiB free
load_tensors: offloaded 24/33 layers to GPU
load_tensors:      CUDA0 model buffer size = 1834.54 MiB
load_tensors:      CUDA_Host model buffer size = 1014.83 MiB
llama_kv_cache:      CUDA0 KV buffer size = 368.00 MiB
sched_reserve:      CUDA0 compute buffer size = 91.00 MiB
```

A.9 Source-level facts (read, not measured)

Fact	Location
Watchdog payload has no <code>options</code> block	<code>llm_watchdog.py:45–51</code>
Watchdog client timeout = 120 s	<code>llm_watchdog.py:55</code>
<code>num_predict</code> values present: 300, 150, 300	<code>api.py:424, 573, 1145</code>
<code>num_ctx</code> : zero occurrences repository-wide	—
<code>stream: False</code> at all 4 call sites	<code>api.py</code> ×3, <code>llm_watchdog.py</code> ×1
4 × <code>requests.post</code> with no <code>timeout</code> argument	<code>api.py:425, 462, 1177, 1232</code>
<code>keep_alive: -1</code> at all call sites (correct)	<code>api.py</code> ×3, <code>llm_watchdog.py</code> ×1
Detoxify constructed without <code>device</code>	<code>toxicity_guard.py:16</code>
Two spaCy models under one <code>lang_code</code>	<code>api.py:49–57</code>
Hardcoded demo keyword interception	<code>api.py:1147–1176</code>
Echo-only second LLM hop, 2 branches	<code>api.py:1200, 1206</code>
System prompt mutated in place before hop 2	<code>api.py:1194</code>
Volatile content interpolated into a system prompt	<code>api.py:537</code>
Fresh <code>AnalyzerEngine()</code> per request	<code>api.py:595</code>
<code>alarms.json</code> full load-and-rewrite per alarm, ×2 files	<code>diff_engine.py:29–38</code>
uvicorn launched with no <code>--workers</code>	<code>start_backend.bat</code>
CPU-only torch installed deliberately	<code>Dockerfile</code> (PyTorch CPU wheel index)
Model names already externalised for a GPU switch	<code>config.py:14–19, config.json</code>

AI Governance Backend — GPU Acceleration & ML/LLM Performance Improvement Plan · 2 September 2026 · Analysis only; no source files were modified. All figures are measured on the machine described in §2.1 or read from the repository at commit `c02d40c`. Items that could not be measured are marked as such.